

Introducción a la Complejidad Temporal

Lectura complementaria: <https://www.geeksforgeeks.org/dsa/complete-guide-on-complexity-analysis/>

La complejidad temporal de un algoritmo se define como la cantidad de tiempo que tarda en ejecutarse en función del tamaño de la entrada (n). Representa el crecimiento del tiempo de ejecución respecto a la entrada, no el tiempo real en una máquina específica.

Notación Big O:

$$O(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n), \forall n \geq n_0\}$$

Asumiremos que las operaciones básicas (comparaciones, asignaciones, retornos y lectura de variables) toman tiempo constante: $O(1)$. El tiempo total es la suma de los tiempos de cada declaración.

Ejemplos Básicos

Ejemplo 1

Enunciado: Verificar si un número N es par o impar.

Respuesta:

```
public static void checkEvenOdd(int N) {  
    int r = N % 2;  
    if (r == 0) {  
        System.out.println("Even");  
    } else {  
        System.out.println("Odd");  
    }  
}
```

Ejemplo 2

Enunciado: Implementar búsqueda binaria recursiva para encontrar la posición de un elemento x en un arreglo ordenado. Retornar -1 si no se encuentra.

Respuesta:

```
static int binarySearch(int arr[], int l, int r, int x) {  
    if (r >= l) {  
        int mid = l + (r - l) / 2;  
        if (arr[mid] == x) return mid;  
        if (arr[mid] > x) return binarySearch(arr, l, mid - 1, x);  
        return binarySearch(arr, mid + 1, r, x);  
    }  
    return -1;  
}
```

Ejemplo 3

Enunciado: Búsqueda lineal para encontrar la posición de un elemento x en un arreglo. Retornar -1 si no se encuentra.

Respuesta:

```
static int search(int[] arr, int N, int x) {
    for (int i = 0; i < N; i++) {
        if (arr[i] == x) {
            return i;
        }
    }
    return -1;
}
```

Ejemplo 4

Enunciado: Dado un arreglo A y un valor x , determinar si existe un par de elementos en el arreglo cuya suma sea exactamente x .

Respuesta:

```
static boolean chkPair(int[] A, int x) {
    int size = A.length;
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (A[i] + A[j] == x) {
                return true;
            }
        }
    }
    return false;
}
```

Ejercicios de Análisis

Ejercicio 1: Analice el siguiente código y determine su complejidad temporal.

```
for (int i = 0; i < n; i++) {
    // Operaciones  $O(1)$ 
}
for (int j = 0; j < m; j++) {
    // Operaciones  $O(1)$ 
}
```

Respuesta: La complejidad temporal es $O(n + m)$.

Ejercicio 2: Analice el siguiente código y determine su complejidad temporal.

```
while (n > 0) {
    n = n / 2;
    // Operaciones  $O(1)$ 
}
```

Respuesta: La complejidad temporal es $O(\log n)$.

Ejercicio 3: Analice el siguiente código y determine su complejidad temporal.

```
for (int i = 0; i < 1000; i += 10) {
    // Operaciones  $O(1)$ 
}
```

Respuesta: La complejidad temporal es $O(1)$, ya que el ciclo se ejecuta una cantidad constante de veces (100 iteraciones).

Competencia

Problema 1: Subtract the Product and Sum of Digits of an Integer

Enlace: <https://leetcode.com/problems/subtract-the-product-and-sum-of-digits-of-an-integer/description/>

Enunciado: Dado un número entero n , devuelva la diferencia entre el producto de sus dígitos y la suma de sus dígitos.

Ejemplo:

- **Entrada:** $n = 234$
- **Salida:** 15
- **Explicación:** Producto = $2 \times 3 \times 4 = 24$, Suma = $2 + 3 + 4 = 9$. Resultado = $24 - 9 = 15$.

Respuesta:

```
class Solution {
    public int subtractProductAndSum(int n) {
        int productVal = 1;
        int sumVal = 0;

        while (n > 0) {
            int digit = n % 10;
            productVal *= digit;
            sumVal += digit;
            n /= 10;
        }

        return productVal - sumVal;
    }
}
```

Complejidad: Temporal $O(\log_{10} n)$, Espacial $O(1)$.

Problema 2: Valid Palindrome

Enlace: <https://leetcode.com/problems/valid-palindrome/description/>

Enunciado: Una frase es un palíndromo si, tras convertir todo a minúsculas y eliminar los caracteres no alfanuméricos, se lee igual hacia adelante y hacia atrás. Dado un string s , devuelva `true` si es un palíndromo, o `false` de lo contrario.

Ejemplo:

- **Entrada:** $s = \text{"A man, a plan, a canal: Panama"}$
- **Salida:** true
- **Explicación:** Eliminando caracteres no alfanuméricos queda `"amanaplanacanalpanama"`, el cual es un palíndromo.

Respuesta:

```
class Solution {
    public boolean isPalindrome(String s) {
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
```

```

        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
            left++;
        }
        while (left < right && !Character.isLetterOrDigit(s.charAt(right)))
        {
            right--;
        }

        char leftChar = Character.toLowerCase(s.charAt(left));
        char rightChar = Character.toLowerCase(s.charAt(right));

        if (leftChar != rightChar) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}

```

Complejidad: Temporal $O(n)$, Espacial $O(1)$.

Problema 3: Move Zeroes

Enlace: <https://leetcode.com/problems/move-zeroes/description/>

Enunciado: Dado un arreglo de enteros `nums`, mueva todos los 0 al final mientras mantiene el orden relativo de los elementos distintos de cero. Esto debe hacerse *in-place* sin realizar una copia del arreglo.

Ejemplo:

- **Entrada:** `nums = [0,1,0,3,12]`
- **Salida:** `[1,3,12,0,0]`

Respuesta:

```

class Solution {
    public void moveZeroes(int[] nums) {
        int insertPos = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                nums[insertPos] = nums[i];
                insertPos++;
            }
        }

        while (insertPos < nums.length) {
            nums[insertPos] = 0;
            insertPos++;
        }
    }
}

```

Complejidad: Temporal $O(n)$, Espacial $O(1)$.